Probability and Expected Value in Computational Problem Solving: A Formal Analysis of Probability Dynamic Programming, Absorbing Markov Chains, Randomized Sampling, and LeetCode Algorithmic Paradigms

Abstract

This paper presents a formal, comprehensive research analysis of probability, expected value, and randomized algorithms in competitive programming and computer science, focusing on canonical LeetCode problem domains. First, we formalize the Kolmogorov Axioms, Linearity of Expectation, Conditional Probability, and the Law of Total Expectation. Second, we dissect Probability Dynamic Programming (Probability DP), analyzing spatial grid transitions (Knight Probability in Chessboard - LC 688), continuous-to-discrete state collapses with asymptotic cutoff proofs (Soup Servings - LC 808), sliding window probability accelerations (New 21 Game - LC 837), and non-homogeneous coin chains (Toss Strange Coins - LC 1230). Third, we model Expected Value DP and Absorbing Markov Chains using First-Step Analysis, proving the harmonic expectation bound $\mathbb{E}[T] = nH_n$ for the Coupon Collector problem, deriving $E_k = 2^{k+1} - 2$ for consecutive heads, and proving structural invariance for Airplane Seat Assignment (LC 1227). Fourth, we rigorously analyze randomized sampling protocols, proving reservoir sampling uniformity $P = 1/N$ (LC 382/398), rejection sampling expectations (Rand10 from Rand7 - LC 470), geometric polar transformations (Random Point in Circle - LC 478), weighted sampling (LC 528), and Fisher-Yates uniform shuffle (LC 384). Production-grade, compilable Java implementations are provided for all algorithms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Mathematical Foundations, Axiomatic Definitions, and Reference Guide

To establish absolute mathematical rigor and provide an accessible entry point for researchers, computer scientists, and software engineers, this section formalizes the core probabilistic concepts, random variable properties, asymptotic notations, and symbol conventions used throughout this research paper.

1.1 Probability Spaces and Kolmogorov Axioms

A formal probability space is defined as a triple $(\Omega, \mathcal{F}, P)$, where:

1) Ω is the sample space, representing the set of all possible outcomes of a random experiment.
2) $\mathcal{F}$ is a σ-algebra of subsets of Ω, representing the set of all measurable events.
3) $P : \mathcal{F} \to [0, 1]$ is the probability measure satisfying Kolmogorov's Axioms:
   - Non-negativity: $P(A) \geq 0$ for all $A \in \mathcal{F}$.
   - Normalization: $P(\Omega) = 1$.
   - Countable Additivity: For any sequence of pairwise disjoint events $A_1, A_2, \cdots \in \mathcal{F}$:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

1.2 Random Variables and Expectation Algebra

Let $X : \Omega \to \mathbb{R}$ be a discrete random variable taking values in a countable set $\mathcal{X}$.

Definition 1 (Expected Value). The expected value (or mean) of X, denoted $\mathbb{E}[X]$, is the probability-weighted sum of all possible values:

$$\mathbb{E}[X] = \sum_{x \in \mathcal{X}} x \cdot P(X = x) \tag{1}$$

provided the sum is absolutely convergent ($\sum_x |x|P(X=x) < \infty$).

Theorem 2 (Linearity of Expectation). For any collection of discrete random variables $X_1, X_2, \ldots, X_n$ defined on the same probability space and any real constants $c_1, c_2, \ldots, c_n$:

$$\mathbb{E}\left[\sum_{i=1}^{n} c_i X_i\right] = \sum_{i=1}^{n} c_i \, \mathbb{E}[X_i] \tag{2}$$

Crucially, Eq. (2) holds unconditionally, regardless of whether the random variables X_i are pairwise independent or dependent.

1.3 Conditional Probability and Conditioning Theorems

For two events $A, B \in \mathcal{F}$ with $P(B) > 0$, the conditional probability of A given B is defined as $P(A|B) = \frac{P(A \cap B)}{P(B)}$.

Theorem 3 (Law of Total Probability). Let $\{B_1, B_2, \ldots, B_k\}$ be a partition of the sample space Ω such that $P(B_i) > 0$ for all i and $\bigcup_{i=1}^{k} B_i = \Omega$. Then for any event A:

$$P(A) = \sum_{i=1}^{k} P(A|B_i) \cdot P(B_i) \tag{3}$$

Theorem 4 (Law of Total Expectation). For random variables X and Y defined on the same probability space:

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X|Y]\big] = \sum_{y} \mathbb{E}[X|Y=y] \cdot P(Y=y) \tag{4}$$

TABLE 1
Quick Reference Table: Mathematical Symbols, Operators, and Algorithmic Domains

Symbol	Name / Meaning	Algorithmic Context & Domain
$P(A)$	Probability of Event A	Likelihood of state transitions or problem conditions.
$\mathbb{E}[X]$	Expected Value of X	Mean steps, draws, or flips until reaching target states.
$P(A\|B)$	Conditional Probability	Transition probabilities in Dynamic Programming grids.
$\mathbf{I}_A$	Indicator Variable ($\mathbf{I}_A \in \{0,1\}$)	Counts occurrence of event A ($\mathbb{E}[\mathbf{I}_A] = P(A)$).
$dp[i][j]$	Probabilistic State Entry	Probability of reaching state (i,j) or game score j.
$\mathcal{U}(a,b)$	Uniform Discrete Distribution	Selecting integers randomly from $[a,b]$ with equal probability.
H_n	n-th Harmonic Number	$\sum_{i=1}^{n} \frac{1}{i} \approx \ln n + \gamma$; Coupon Collector expectation bound.
$a \pmod M$	Modular Arithmetic	Wrapping large combinatorial integer sums modulo 10^9+7.

2 Probability Dynamic Programming (Probability DP)

In competitive programming and software engineering interviewing (exemplified by LeetCode), many probabilistic questions require populating an n-dimensional state grid where each cell $dp[i][j]$ represents the probability of residing in state (i,j) or reaching a terminal target condition.

2.1 Spatial Grid Probability: Knight Probability in Chessboard (LC 688)

2.1.1 Formal Problem Definition

Given an $N \times N$ chessboard, a knight starts at cell (r_0, c_0) and makes exactly K moves. At each step, the knight chooses one of eight valid L-shaped moves uniformly at random with probability $p = 1/8$. If the knight moves off the board, it remains off the board. We calculate the probability that the knight remains on the board after K moves.

2.1.2 Mathematical Recurrence Formulation

Let $dp[k][r][c]$ denote the probability that the knight is positioned at row r and column c after making k moves.

Let $\Delta = \{(-2,-1), (-2,1), (-1,-2), (-1,2), (1,-2), (1,2), (2,-1), (2,1)\}$ be the set of 8 knight moves. For a knight at cell (r', c') at move $k-1$, its probability mass is distributed equally $(1/8)$ among its valid successors. Reversing this top-down view to a bottom-up view, the probability of being at (r,c) at move k is the sum of probabilities of being at valid previous cells $(r - dr, c - dc)$ at move $k-1$:

$$dp[k][r][c] = \frac{1}{8} \sum_{(dr,dc) \in \Delta} dp[k-1][r-dr][c-dc] \tag{5}$$

where $dp[k-1][r-dr][c-dc] = 0$ if $(r-dr, c-dc)$ lies outside the $N \times N$ board boundaries.

2.1.3 Base Case and Total Probability

- Base Case ($k = 0$): $dp[0][r_0][c_0] = 1.0$, and $dp[0][r][c] = 0.0$ for all $(r, c) \neq (r_0, c_0)$.
- Total Survival Probability after K moves:

$$P_{\text{survive}} = \sum_{r=0}^{N-1} \sum_{c=0}^{N-1} dp[K][r][c]$$

2.1.4 Space Optimization and Complexity Proof

Since $dp[k][r][c]$ depends exclusively on $dp[k-1][r][c]$, storing the full 3D table of size $K \times N \times N$ is unnecessary. We maintain two 2D matrices ('prev' and 'curr') of size $N \times N$, reducing space complexity from $O(K \cdot N^2)$ to $O(N^2)$. Total time complexity is $\Theta(K \cdot N^2 \cdot 8) = \Theta(K \cdot N^2)$.

2.1.5 Production-Grade Java Implementation

```
public class KnightProbability {
    private static final int[][] MOVES = {
        {-2, -1}, {-2, 1}, {-1, -2}, {-1, 2},
        {1, -2}, {1, 2}, {2, -1}, {2, 1}
    };

    public static double knightProbability(int n, int k, int row, int column) {
        double[][] prev = new double[n][n];
        prev[row][column] = 1.0;

        for (int step = 1; step <= k; step++) {
            double[][] curr = new double[n][n];
            for (int r = 0; r < n; r++) {
                for (int c = 0; c < n; c++) {
                    if (prev[r][c] > 0) {
                        for (int[] move : MOVES) {
                            int nr = r + move[0];
                            int nc = c + move[1];
                            if (nr >= 0 && nr < n && nc >= 0 && nc < n) {
                                curr[nr][nc] += prev[r][c] / 8.0;
                            }
                        }
                    }
                }
            }
            prev = curr;
        }

        double totalProb = 0.0;
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                totalProb += prev[r][c];
            }
        }
        return totalProb;
    }
}
```

2.2 Continuous State Truncation: Soup Servings (LC 808)

2.2.1 Formal Problem Definition

There are two types of soup, A and B, initially with N mL each. We serve four operations chosen uniformly at random with probability 0.25:

1) Serve 100 mL of A and 0 mL of B.
2) Serve 75 mL of A and 25 mL of B.
3) Serve 50 mL of A and 50 mL of B.
4) Serve 25 mL of A and 75 mL of B.

If a serving calls for more soup than available, we serve whatever remains. The process stops when either A or B runs out. We return the probability that A runs out first plus $0.5\times$ (probability that A and B run out simultaneously).

2.2.2 Discretization and State Space Recurrence

Since all serving volumes are multiples of 25 mL, we discretize quantities by dividing by 25: $n = \lceil N/25.0 \rceil$. The four serving operations become: $(4, 0)$, $(3, 1)$, $(2, 2)$, $(1, 3)$.

Let $dp(a, b)$ be the required probability when a units of A and b units of B remain.

$$dp(a,b) = 0.25 \cdot \Big(dp(a-4,b) + dp(a-3,b-1) + dp(a-2,b-2) + dp(a-1,b-3) \Big) \tag{6}$$

2.2.3 Terminal Base Conditions

- $dp(a, b) = 0.5$ if $a \leq 0$ and $b \leq 0$ (A and B run out simultaneously).
- $dp(a, b) = 1.0$ if $a \leq 0$ and $b > 0$ (A runs out first).
- $dp(a, b) = 0.0$ if $a > 0$ and $b \leq 0$ (B runs out first).

2.2.4 Formal Asymptotic Cutoff Proof

Theorem 5 (Asymptotic Convergence of Soup Servings). As $N \to \infty$, the probability $P(N) \to 1.0$. Specifically, for $N \geq 4800$ (discretized $n \geq 192$), $1.0 - P(N) < 10^{-5}$. Thus, for $N \geq 4800$, the algorithm can return 1.0 in $O(1)$ time.

Proof. Consider the expected change in soup quantities per operation: $\mathbb{E}[\Delta A] = \frac{100+75+50+25}{4} = 62.5$ mL, whereas $\mathbb{E}[\Delta B] = \frac{0+25+50+75}{4} = 37.5$ mL. Because $\mathbb{E}[\Delta A] = 62.5 > \mathbb{E}[\Delta B] = 37.5$, soup A is depleted at a significantly faster expected rate than soup B (an average net drift of 25 mL per turn in favor of depleting A). By the Central Limit Theorem and Law of Large Numbers, as N increases, the probability distribution of (A_k, B_k) concentrates tightly around the mean trajectory where A reaches zero long before B. For $N \geq 4800$, the probability that B runs out first or simultaneously is less than 10^{-5}, which rounds to 1.0 under double-precision floating point accuracy. □

2.2.5 Production-Grade Java Implementation

```
public class SoupServings {
    private static double[][] memo;

    public static double soupServings(int n) {
        if (n >= 4800) return 1.0;
        int units = (int) Math.ceil(n / 25.0);
        memo = new double[units + 1][units + 1];
        return helper(units, units);
    }

    private static double helper(int a, int b) {
        if (a <= 0 && b <= 0) return 0.5;
        if (a <= 0) return 1.0;
        if (b <= 0) return 0.0;
        if (memo[a][b] > 0) return memo[a][b];

        double prob = 0.25 * (
            helper(a - 4, b) +
            helper(a - 3, b - 1) +
            helper(a - 2, b - 2) +
            helper(a - 1, b - 3)
        );
        memo[a][b] = prob;
        return prob;
    }
}
```

2.3 Sliding Window Probability Optimization: New 21 Game (LC 837)

2.3.1 Formal Problem Definition

Alice plays a game starting with 0 points. While her total score is strictly less than K points, she draws a number independently and uniformly at random from $[1, W]$. The game ends when her score reaches or exceeds K. We determine the probability that her final score is less than or equal to N.

2.3.2 Mathematical Recurrence and Range Sums

Let $dp[i]$ denote the probability of obtaining a total score of exactly i points during the game.

For any score $i \leq N$, Alice could have reached i from a prior score $j < K$ by drawing card $i - j \in [1, W]$. Thus, for $i \geq 1$:

$$dp[i] = \frac{1}{W} \sum_{j=\max(0,i-W)}^{\min(i-1,K-1)} dp[j] \tag{7}$$

2.3.3 Sliding Window Acceleration from $O(N \cdot W)$ to $O(N)$

Evaluating Eq. (7) naively takes $O(W)$ time per score state i, yielding $O(N \cdot W)$ total time. Notice that the summation $\sum_j dp[j]$ covers a sliding window of size at most W ending at $\min(i-1, K-1)$. We maintain a running window sum W_{sum}:

- $dp[i] = \frac{W_{\text{sum}}}{W}$.
- If $i < K$, we add $dp[i]$ to W_{sum} (since future draws can originate from score i).
- If $i - W \geq 0$, we subtract $dp[i-W]$ from W_{sum} (since score $i-W$ drops out of the sliding window).

This maintains W_{sum} in $O(1)$ time per step, achieving optimal $O(N)$ linear time and $O(N)$ space.

2.3.4 Production-Grade Java Implementation

```
public class New21Game {
    public static double new21Game(int n, int k, int maxPts) {
        if (k == 0 || n >= k + maxPts) return 1.0;

        double[] dp = new double[n + 1];
        dp[0] = 1.0;
        double windowSum = 1.0;
        double result = 0.0;

        for (int i = 1; i <= n; i++) {
            dp[i] = windowSum / maxPts;

            if (i < k) {
                windowSum += dp[i];
            } else {
                result += dp[i]; // Score reached terminal threshold >= K
            }

            if (i - maxPts >= 0) {
                windowSum -= dp[i - maxPts];
            }
        }
        return result;
    }
}
```

2.4 Non-Homogeneous Coin Toss Chains: Toss Strange Coins (LC 1230)

2.4.1 Formal Problem Definition

You have N independent coins, where the i-th coin has probability $prob[i]$ of facing heads. You toss all N coins. Return the probability that the total number of coins facing heads is exactly $target$.

2.4.2 2D to 1D DP State Transition

Let $dp[i][j]$ be the probability of getting exactly j heads after tossing the first i coins. At coin i (1-indexed):

1) Coin i comes up Heads (probability p_i): requires $j-1$ heads from first $i-1$ coins ($dp[i-1][j-1] \cdot p_i$).
2) Coin i comes up Tails (probability $1-p_i$): requires j heads from first $i-1$ coins ($dp[i-1][j] \cdot (1-p_i)$).

Combining choices yields the fundamental recurrence:

$$dp[i][j] = dp[i-1][j-1] \cdot p_i + dp[i-1][j] \cdot (1-p_i) \tag{8}$$

By iterating j backwards from $target$ down to 0, we optimize space to a 1D array of size $target+1$, achieving $O(N \cdot target)$ time and $O(target)$ space.

2.4.3 Production-Grade Java Implementation

```
public class TossStrangeCoins {
    public static double probabilityOfHeads(double[] prob, int target) {
        double[] dp = new double[target + 1];
        dp[0] = 1.0; // Base case: 0 coins tossed, 0 heads

        for (double p : prob) {
            for (int j = Math.min(target, prob.length); j >= 0; j--) {
```

```
            double heads = (j > 0) ? dp[j - 1] * p : 0.0;
            double tails = dp[j] * (1.0 - p);
            dp[j] = heads + tails;
        }
      }
      return dp[target];
   }
}
```

3 Expected Value DP and Absorbing Markov Chains

Expected value calculations in discrete algorithmic state spaces are modeled using First-Step Analysis on Absorbing Markov Chains.

3.1 First-Step Analysis and Linear Systems

Let S be a state space. For a state $s \in S$, let $E(s) = \mathbb{E}[\text{Cost to reach terminal state } t \mid \text{start at } s]$. By the Law of Total Expectation, conditioning on the first step taken from s:

$$E(s) = 1 + \sum_{s' \in S} P(s \to s') \cdot E(s') \tag{9}$$

with boundary conditions $E(t) = 0$ for all terminal absorbing states t.

3.2 Coupon Collector's Problem and Harmonic Bound Proof

Theorem 6 (Coupon Collector Expectation). Suppose there are n distinct coupon types. At each step, a coupon is drawn independently and uniformly at random with replacement. The expected total number of draws T required to collect at least one coupon of each of the n types is:

$$\mathbb{E}[T] = nH_n = n \sum_{i=1}^{n} \frac{1}{i} = n \ln n + \gamma n + O(1) \tag{10}$$

where H_n is the n-th harmonic number and $\gamma \approx 0.577215$ is the Euler-Mascheroni constant.

Proof. Let t_i be the number of additional draws needed to collect the i-th new coupon after $(i-1)$ distinct coupons have already been collected. The total time is $T = \sum_{i=1}^{n} t_i$. When $(i-1)$ unique coupons are owned, the probability of drawing a new uncollected coupon on any single draw is:

$$p_i = \frac{n - (i-1)}{n} = \frac{n - i + 1}{n}$$

The random variable t_i follows a Geometric distribution with parameter p_i. The expected value of a Geometric random variable is $\mathbb{E}[t_i] = \frac{1}{p_i} = \frac{n}{n-i+1}$. By Linearity of Expectation:

$$\mathbb{E}[T] = \mathbb{E}\left[\sum_{i=1}^{n} t_i\right] = \sum_{i=1}^{n} \mathbb{E}[t_i] = \sum_{i=1}^{n} \frac{n}{n-i+1} = n\left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{1}\right) = nH_n$$

Using integral approximation bounds $\ln n < H_n < \ln n + 1$, $\mathbb{E}[T] = \Theta(n \ln n)$. □

3.3 Consecutive Heads Expectation Recurrence

Theorem 7 (Expected Flips for k Consecutive Heads). Let E_k be the expected number of fair coin flips needed to obtain k consecutive heads. Then:

$$E_k = 2^{k+1} - 2 \tag{11}$$

Proof. Consider the state after obtaining $k-1$ consecutive heads (which took E_{k-1} expected flips on average). We flip the coin once more:

1) With probability $1/2$, we get Heads: we reach k consecutive heads. Total flips $= E_{k-1} + 1$.
2) With probability $1/2$, we get Tails: our streak resets to 0. Total flips $= E_{k-1} + 1 + E_k$.

Conditioning on this single flip:

$$E_k = (E_{k-1} + 1) + \frac{1}{2}(0) + \frac{1}{2}(E_k) \implies \frac{1}{2}E_k = E_{k-1} + 1 \implies E_k = 2E_{k-1} + 2$$

Solving this linear recurrence with base case $E_0 = 0$:

$$E_1 = 2(0) + 2 = 2, \quad E_2 = 2(2) + 2 = 6, \quad E_3 = 2(6) + 2 = 14, \quad \ldots \quad E_k = 2^{k+1} - 2$$

□

3.4 Invariant Structural Expectation: Airplane Seat Assignment (LC 1227)

Theorem 8 (Airplane Seat Probability Invariance). N passengers board an airplane with N assigned seats. The first passenger loses their ticket and picks a seat uniformly at random. Each subsequent passenger takes their own seat if available, or picks an available seat uniformly at random if occupied. The probability that the N-th passenger gets their own seat is 1.0 for $N = 1$, and exactly 0.5 for all $N \geq 2$.

Proof. Let $P(N)$ be the probability for N passengers. For $N = 1$, $P(1) = 1.0$. For $N \geq 2$, consider the first passenger's choice among N seats:

- Choice 1: Takes Seat 1 (probability $1/N$). Passenger 1 gets own seat; every subsequent passenger gets their own seat. Passenger N gets own seat (Success).
- Choice 2: Takes Seat N (probability $1/N$). Passenger N's seat is taken immediately (Failure).
- Choice 3: Takes Seat $k \in \{2, \ldots, N-1\}$ (probability $1/N$ each). Passengers $2 \ldots k-1$ take their own seats. Passenger k arrives, finds seat occupied, and faces identical choice among remaining seats. This reduces the problem to an instance of size $N - k + 1$.

By Law of Total Probability:

$$P(N) = \frac{1}{N}(1) + \frac{1}{N}(0) + \frac{1}{N}\sum_{k=2}^{N-1} P(N-k+1) = \frac{1}{N} + \frac{1}{N}\sum_{m=2}^{N-1} P(m)$$

For $N = 2$: $P(2) = 1/2$. By induction, assuming $P(m) = 1/2$ for all $m < N$:

$$P(N) = \frac{1}{N} + \frac{1}{N}(N-2)\left(\frac{1}{2}\right) = \frac{1}{N} + \frac{N-2}{2N} = \frac{2+N-2}{2N} = \frac{N}{2N} = \frac{1}{2}$$

□

4 Randomized Algorithms and Uniform Sampling Protocols

4.1 Stream Sampling: Reservoir Sampling (LC 382 & LC 398)

4.1.1 Formal Problem Definition

Given a singly linked list or data stream of unknown/infinite length N, pick a random node's value such that every node has an equal probability $1/N$ of being chosen, using $O(1)$ auxiliary space and a single pass.

4.1.2 Algorithm Description

Initialize 'chosen = head.val'. Iterate through the stream with 1-based index i: For the i-th element, generate a random integer in $[1, i]$. If the generated number is 1 (which occurs with probability $1/i$), replace 'chosen' with the current element's value.

4.1.3 Mathematical Proof of Uniform Probability

Theorem 9 (Reservoir Sampling Uniformity). After processing N elements, every element $k \in \{1, 2, \ldots, N\}$ resides in the reservoir with probability exactly $\frac{1}{N}$.

Proof. For element k to be the final selected item after N steps, two independent events must occur:

1) Element k must be selected when it is processed at step k (Probability $= \frac{1}{k}$).
2) Element k must NOT be replaced by any subsequent element j for all $j = k+1, k+2, \ldots, N$.

At step j, the probability of NOT replacing the current reservoir item is $1 - \frac{1}{j} = \frac{j-1}{j}$.

Multiplying the probabilities of these independent events:

$$P(\text{element } k \text{ chosen at end}) = \frac{1}{k} \times \prod_{j=k+1}^{N}\left(1 - \frac{1}{j}\right) = \frac{1}{k} \times \left(\frac{k}{k+1} \times \frac{k+1}{k+2} \times \cdots \times \frac{N-1}{N}\right)$$

Notice that the telescoping product cancels out all intermediate numerators and denominators from k to $N-1$:

$$P(\text{element } k \text{ chosen at end}) = \frac{1}{k} \times \frac{k}{N} = \frac{1}{N}$$

□

4.1.4 Production-Grade Java Implementation

```
import java.util.Random;

public class ReservoirSampling {
    private ListNode head;
    private Random rand;

    public ReservoirSampling(ListNode head) {
        this.head = head;
        this.rand = new Random();
    }

    public int getRandom() {
        ListNode curr = head;
        int chosenVal = curr.val;
        int i = 1;

        while (curr != null) {
            // Pick current element with probability 1/i
            if (rand.nextInt(i) == 0) {
                chosenVal = curr.val;
            }
            curr = curr.next;
            i++;
        }
        return chosenVal;
    }
}
```

4.2 Rejection Sampling & Geometric Transformations

4.2.1 Rand10() Using Rand7() (LC 470)

To generate a uniform integer in $[1, 10]$ using a $Rand7()$ function, map two calls of $Rand7()$ into a 2D matrix of size $7 \times 7 = 49$:

$$idx = 7 \cdot (Rand7() - 1) + (Rand7() - 1) \quad \in [0, 48]$$

If $idx < 40$, return $(idx \bmod 10) + 1$. If $idx \geq 40$, reject and retry.

Theorem 10 (Expected Retries for Rand10). The number of trials until acceptance follows a Geometric distribution with parameter $p = 40/49$. The expected number of $Rand7()$ calls per $Rand10()$ output is $2 \times \frac{49}{40} = 2.45$ calls.

4.2.2 Random Point in a Circle (LC 478)

To pick a point uniformly at random inside a circle of radius R centered at (x_0, y_0):

- Naive Flaw: Generating $\theta \sim \mathcal{U}(0, 2\pi)$ and $r \sim \mathcal{U}(0, R)$ causes points to cluster near the center because area elements scale as $r\, dr\, d\theta$.
- Correct Inverse CDF Method: Set $\theta = 2\pi U_1$ and $r = R\sqrt{U_2}$, where $U_1, U_2 \sim \mathcal{U}(0, 1)$. The square root $\sqrt{U_2}$ accounts for the quadratic area growth $A \propto r^2$.

4.2.3 Production-Grade Java Implementation for Rand10

```
public class Rand10FromRand7 {
    // Mock rand7() API provided by LeetCode
    private int rand7() {
        return (int) (Math.random() * 7) + 1;
    }

    public int rand10() {
        int row, col, idx;
        do {
            row = rand7();
            col = rand7();
            idx = 7 * (row - 1) + (col - 1); // 0 to 48
        } while (idx >= 40); // Reject top 9 values (40 to 48)
```

```
        return (idx % 10) + 1;
    }
}
```

4.3 Discrete Distribution Sampling: Weighted Random Pick (LC 528)

4.3.1 Prefix Sums + Binary Search Protocol

Given positive weights $W = [w_0, w_1, \ldots, w_{N-1}]$, sample index i with probability $\frac{w_i}{\sum w_k}$.

1) Construct a prefix sum array P, where $P[i] = \sum_{j=0}^{i} w_j$. Total weight $T = P[N-1]$.
2) Draw a random target integer $target \sim \mathcal{U}(1, T)$.
3) Binary search for the smallest index i such that $P[i] \geq target$.

Time Complexity: Preprocessing $O(N)$, Pick Query $O(\log N)$. Space: $O(N)$.

4.3.2 Production-Grade Java Implementation

```
import java.util.Random;

public class WeightedRandomPick {
    private int[] prefixSums;
    private int totalSum;
    private Random rand;

    public WeightedRandomPick(int[] w) {
        this.prefixSums = new int[w.length];
        this.rand = new Random();
        int sum = 0;
        for (int i = 0; i < w.length; i++) {
            sum += w[i];
            prefixSums[i] = sum;
        }
        this.totalSum = sum;
    }

    public int pickIndex() {
        int target = rand.nextInt(totalSum) + 1; // 1 to totalSum
        int low = 0, high = prefixSums.length - 1;

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (prefixSums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }
}
```

4.4 Permutation Sampling: Fisher-Yates Shuffle (LC 384)

Theorem 11 (Fisher-Yates Uniform Permutation). The Fisher-Yates shuffle algorithm generates every possible permutation of an N-element array with equal probability $\frac{1}{N!}$ in $\Theta(N)$ time and $O(1)$ auxiliary space.

Proof. Iterate index i from $N-1$ down to 1. At step i, pick a random index $j \in [0, i]$ and swap elements at i and j. The number of possible choices made by the algorithm is $N \times (N-1) \times \cdots \times 1 = N!$. Each sequence of choices produces a unique permutation. Since each choice is uniform, all $N!$ permutations are generated with probability $\frac{1}{N!}$. □

4.4.1 Production-Grade Java Implementation

```
import java.util.Random;

public class FisherYatesShuffle {
    private int[] original;
    private Random rand;

    public FisherYatesShuffle(int[] nums) {
        this.original = nums.clone();
        this.rand = new Random();
    }

    public int[] reset() {
        return original.clone();
    }

    public int[] shuffle() {
        int[] array = original.clone();
        for (int i = array.length - 1; i > 0; i--) {
            int j = rand.nextInt(i + 1); // Random index from 0 to i
            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
        return array;
    }
}
```

5 Comparative Algorithmic Taxonomy & Empirical Synthesis

Table 2 provides a synthesis of all major LeetCode probability, expected value, and sampling problems analyzed in this paper.

TABLE 2
Taxonomy of LeetCode Probability, Expected Value, and Sampling Algorithms

LeetCode Problem / Category	LC #	Core Mathematical Concept	Time	Space
Knight Probability in Chessboard	688	3D Probability DP / Grid Transitions	$O(K \cdot N^2)$	$O(N^2)$
Soup Servings	808	Asymptotic Cutoff DP ($N \geq 4800$)	$O(1)$	$O(1)$
New 21 Game	837	Sliding Window Probability DP	$O(N)$	$O(N)$
Toss Strange Coins	1230	Non-Homogeneous Coin DP	$O(N \cdot K)$	$O(K)$
Airplane Seat Assignment	1227	Proof by Induction ($P = 0.5$)	$O(1)$	$O(1)$
Linked List Random Node	382	Reservoir Sampling Protocol	$O(N)$	$O(1)$
Implement Rand10() using Rand7()	470	Rejection Sampling ($p = 40/49$)	$O(1)$ exp.	$O(1)$
Random Point in Non-Overlapping Rects	497	Weighted Sampling + Area Uniformity	$O(\log N)$	$O(N)$
Random Pick with Weight	528	Prefix Sums + Binary Search	$O(\log N)$	$O(N)$
Shuffle an Array	384	Fisher-Yates Uniform Permutation	$O(N)$	$O(N)$

6 Conclusion

This research paper presented a formal, unified framework for analyzing probability and expected value problems in computational problem solving. We demonstrated how probability dynamic programming, asymptotic state space truncation, Markov chain first-step analysis, reservoir sampling, and rejection sampling transform complex stochastic processes into deterministic, optimal software implementations.